

Project Report:

Enterprise IT Asset & Security Management (EITASM)

Course:

420-921-VA Database

Professor:

Ms. Silviya Lyubomirova Paskaleva

Group Members:

Danny Gilberto

Nafiseh Khosravi Rad

Kwok Yiu Wong

Date:

July 2, 2026

Table of Contents

1. Project Objective.....	5
1.1. Core Technical Achievements.....	5
1.2. Advanced Database Features	6
1.3. Project Methodology.....	6
2. System Design & Architecture.....	7
2.1. Database Schema Architecture.....	7
2.2. Schema Composition and Data Definitions	7
2.3. Relational Integrity and Constraints	8
2.4. Expectation Outcomes	8
2.5. Functional Requirements	9
2.6. Non-Functional Requirements	9
2.7. Initial Mind Mapping (Data Architecture)	10
2.8. UML Use Case Diagram	10
2.9. Entity-Relationship Diagram (ERD) Logic	10
2.10. Core Entities and Attributes:	11
2.11. Relationships:	11
2.12. Normalization Breakdown (3NF)	12
2.13. Database Logic & Constraints	12
2.14. Implementation for our Trello Board	13
2.15. Data Dictionary & Field Constraints Specifications	13

2.16. Advanced SQL Logic & Automation.....	14
2.17. Technical Stack Architecture.....	15
3.0. Project Roadmap & Execution.....	16
4. Security & Administrative Policy	18
4.1. Backup & Recovery Strategy.....	18
4.2. Management of Authorization (Confidentiality).....	18
4.3. Flow Logic	19
4.4. Security Controls and Audit Procedures	20
4.5. Access Control and Authentication	20
4.6. Monitoring and Incident Response	21
4.7. Data Protection.....	21
5. Technical Environment	22
5.1. Technical Stack Justification & Requirements	22
5.2. Database & Technical Implementation	23
5.3. Design & Modeling Tools.....	23
5.4. Project Management & Methodology.....	23
5.5. Application Architecture (Optional).....	24
5.6. Documentation & Version Control	24
6. Implementation Methodology.....	25
6.1. Database Engineering & Development.....	25
6.2. Technical Standards and Best Practices	25

7. Database Operational Logic.....	27
8. Implementation Checklist	28
9. Team Roles & Responsibilities	29
10. Conclusion and Future Recommendations	30
10.1. Project Conclusion	30
10.2. Future Recommendations	30
APPENDIX	31
I. All tables properly related.....	31
II. Stored Procedures developed for CRUD operations	32
III. Implement at least 1 Custom SQL Function	33
IV. Implement at least 1 SQL View.....	34
V. Class Diagram aligned with Database Design.....	35
VI. Database Administration and Security Controls.....	36
VII. Data Encryption.....	37
VIII. Database Backup	38
IX. Create the Credential using the Storage Account SAS Token	39
X. Force audit event generation	40
XI. Format the logs into a clean table showing.....	41
XII. Role-Based Auditing	42

1. Project Objective

The objective of this project was to design and implement a centralized, secure Relational Database Management System (RDBMS) to track, manage, and secure IT assets for an enterprise of 500 ~ 700 employees. The system successfully bridges the gap between HR, IT, and Security departments to automate asset tracking, software licensing, and security compliance. EITASM moves beyond a simple inventory list by transforming a database into an active security participant that enforces corporate policy at the data layer.

1.1. Core Technical Achievements

- **Asset & License Lifecycle:** Achieved 100% visibility of hardware and software utilization through robust relational modeling.
- **Automated Security Enforcement:** Successfully implemented SQL Triggers to prevent unauthorized asset assignment by comparing Employee clearance levels against Asset security requirements.
- **Data Integrity & Normalization:** The final schema is fully normalized to 3rd Normal Form (3NF), ensuring referential integrity across all Personnel, Inventory, and Incident logs.
- **Administrative Security:** Integrated Transparent Data Encryption (TDE) for encryption-at-rest and established automated auditing to track all structural and data-level modifications.

1.2. Advanced Database Features

- **Confidentiality & RBAC:** Utilized SQL Views and roles to enforce a "Need to Know" principle, ensuring sensitive personnel and security data are restricted.
- **Business Logic Automation:** Developed a suite of Stored Procedures to handle complex CRUD operations, replacing raw SQL in the application layer to enhance security and maintainability.
- **Disaster Recovery:** Implemented an automated backup and recovery strategy, including full, differential, and transaction log backups with a 7-day retention policy to ensure business continuity.

1.3. Project Methodology

The project is managed via **Agile Kanban (Trello)** and version-controlled on **GitHub**. Development is divided into three Sprints:

1. **Sprint 1:** Modeling & Design (ERDs, Use Cases, Mind Mapping).
2. **Sprint 2:** Core Construction (Schema DDL, 700+ record Data Seeding).
3. **Sprint 3:** Advanced Logic (Triggers, Procedures, Security Audit Logging).

2. System Design & Architecture

2.1. Database Schema Architecture

The system utilizes a multi-schema architecture to logically separate concerns between Human Resources (HR), Information Technology (IT), and Security (SEC) departments. The database is structured to maintain strict referential integrity through predefined foreign key constraints.

2.2. Schema Composition and Data Definitions

The database utilizes standardized data types (e.g., int, varchar, date, datetime, bit) to ensure efficient storage and accurate data retrieval.

- **HR Schema (Human Resources):** Manages organizational structure and sensitive employee records.
 - **Departments Table:** Defines organizational units with DepartmentID (int) and ManagerID (int).
 - **Employees Table:** Central registry containing PII, including SocialInsuranceNumber (char, 9), DateOfBirth (date), and UserSecurityLevel (int).
- **IT Schema (Asset Management):** Tracks corporate inventory.
 - **Hardware Assets Table:** Monitors equipment status, including HostName (varchar, 100), PurchaseDate (date), and SecurityRequirement (int).
 - **AssetTypes Table:** Categorizes inventory with TypeName (varchar, 50).
- **SEC Schema (Security & Access):** Controls system integrity and incident tracking.

- **SecurityIncidents Table:** Logs events via SeverityLevelID (int) and Description (varchar, -1).
- **UserCredentials Table:** Manages authentication security with PasswordHash (varchar, 255), AccountLocked (bit), and PasswordExpirationDate (datetime).

2.3. Relational Integrity and Constraints

To maintain data consistency, the following foreign key (FK) constraints have been implemented across the schemas:

Child Schema	Table	Foreign Key Constraint	Referenced Parent
HR	Employees	FK_Employees_Departments	Departments(DepartmentID)
IT	Hardware Assets	FK_HardwareAssets_Employees	Employees(EmployeeID)
IT	Hardware Assets	FK_HardwareAssets_AssetTypes	AssetTypes(AssetTypeID)
SEC	SecurityIncidents	FK_SecurityIncidents_Employees	Employees(EmployeeID)
SEC	SecurityIncidents	FK_SecurityIncidents_HardwareAssets	HardwareAssets(AssetID)
SEC	SecurityIncidents	FK_SecurityIncidents_SeverityLevels	SeverityLevels(SeverityLevelID)
SEC	UserCredentials	FK_UserCredentials_Employees	Employees(EmployeeID)

2.4. Expectation Outcomes

- **Centralized Asset Tracking:** 100% visibility of hardware (laptops, servers) and software licenses.
- **Security Compliance:** Automated triggers to flag unauthorized software or expired security patches.
- **Role-Based Access Control:** Strict data confidentiality utilizing SQL Views and Schemas to ensure departments (HR, IT, Security) only access 'need-to-know' data.
- **Audit Readiness:** Automated SQL scripts to generate compliance reports for stakeholders.

2.5. Functional Requirements

- **FR1: Asset Lifecycle Management:** The system shall allow authorized IT staff to create, update, and decommission hardware assets.
- **FR2: Automated Assignment:** The system must provide a mechanism to link an Employee ID to an Asset ID and a Software License ID simultaneously.
- **FR3: Inventory Alerts:** The system shall generate automated alerts when software license "seats" are 90% utilized or within 30 days of expiration.
- **FR4: Incident Reporting:** Users must be able to log security incidents against specific Serial Numbers, which automatically updates the asset status to "Under Investigation."

2.6. Non-Functional Requirements

- **NFR1: Security & Confidentiality:** Access to the Personnel_Salary and Security_Incidents tables must be restricted via SQL Roles and Views.
- **NFR2: Data Integrity:** The system must enforce referential integrity; for example, an asset cannot be assigned to an Employee ID that does not exist in the Personnel table.
- **NFR3: Availability:** The database must support 24/7 read access for automated security monitoring scripts.
- **NFR4: Recoverability:** The system must include a documented recovery script to restore data to a "last known good state" within 30 minutes of a failure.
- **NFR5: Data Atomicity (ACID):** All asset assignments must be handled as a single transaction; if the hardware assignment fails, the software license provisioning must also roll back.

2.7. Initial Mind Mapping (Data Architecture)

To visualize the scope, the database will be branched into four primary clusters:

- **Personnel:** Employee ID, Name, Department, Role, Security Clearance Level.
- **Hardware Assets:** Serial Number, Asset Type (Workstation/Server/Mobile), Purchase Date, Assigned User, Physical Location.
- **Software & Licenses:** Software Name, License Key, Version, Expiry Date, Seat Count.
- **Security & Incidents:** Incident ID, Severity, Linked Asset, Resolution Status, Timestamp.

2.8. UML Use Case Diagram

The following illustrates the interactions between different enterprise roles and the EITASM system:

- **Employee:** Views assigned assets and reports hardware issues.
- **IT Technician:** Manages inventory, assigns hardware, and installs software.
- **Security Admin:** Reviews security logs, manages clearance levels, and audits compliance.
- **System (Automated):** Triggers alerts for license expirations and clearance mismatches.

2.9. Entity-Relationship Diagram (ERD) Logic

For an enterprise of 700 employees, the database must be normalized to ensure data integrity and prevent redundancy.

2.10. Core Entities and Attributes:

- **Employees (Eliminates transitive dependencies by moving Dept details to the Departments table):** EmployeeID (PK), FirstName, LastName, DepartmentID (FK), Email (Unique), ClearanceLevel (1-5).
- **Departments:** DepartmentID (PK), DeptName, ManagerID (FK to Employees).
- **HardwareAssets:** AssetID (PK), SerialNumber (Unique), Model, AssetTypeID (FK), PurchaseDate, SecurityRequirement (1-5), CurrentStatus, AssignedToEmployeeID (FK).
- **SoftwareLicenses:** LicenseID (PK), SoftwareName, Version, TotalSeats, ExpiryDate.
- **LicenseAssignments:** AssignmentID (PK), LicenseID (FK), EmployeeID (FK), InstallDate.
- **SecurityIncidents:** IncidentID (PK), AssetID (FK), ReporterID (FK to Employees), SeverityLevel (FK - lookup table for Low/Med/High), Description, DateReported, ResolutionStatus (Open, Investigating, Resolved), CreatedAt.

2.11. Relationships:

- **One-to-Many:** One Department has many Employees.
- **One-to-Many:** One Employee can be assigned multiple Hardware Assets.
- **Many-to-Many:** Employees and Software Licenses (resolved via the LicenseAssignments junction table).
- **One-to-Many:** One Hardware Asset can have multiple Security Incidents over its lifecycle.

2.12. Normalization Breakdown (3NF)

- **1st Normal Form (1NF):** Each field contains atomic values (e.g., `FirstName` and `LastName` are separate), and every table has a Primary Key.
- **2nd Normal Form (2NF):** All non-key attributes are fully functional dependent on the Primary Key. By using junction tables like `LicenseAssignments`, we ensure that software details aren't duplicated for every employee.
- **3rd Normal Form (3NF):** We have removed **transitive dependencies**.

Example #1: In the `HardwareAssets` table, we store a `VendorID` rather than the Vendor's name and support contact info. This ensures that if a vendor changes their support number, we update it once in the `Vendors` table, satisfying 3NF by removing transitive dependencies.

Example #2: `SeverityLevel` is moved to a lookup table so that the description of "High Severity" isn't repeated across 500 incident rows.

2.13. Database Logic & Constraints

The flow chart below illustrates the lifecycle of an asset from procurement to decommissioning, including the automated security checks required by our course.

The Operational Flow:

1. **Procurement:** A new laptop is added to `HardwareAssets`.
2. **Onboarding (Agile Sprint 2 Task):** When a new employee is hired, a **Stored Procedure** checks for an "Available" asset.

3. **Security Check (Constraint/Trigger):** Before the assignment is finalized, a **Trigger** verifies the `ClearanceLevel`. If the employee is Level 1 but the asset is marked for Level 3, the transaction fails.
4. **Maintenance/Incident:** If a `SecurityIncident` is logged against an `AssetID`, the `HardwareAssets.CurrentStatus` is automatically updated to "Under Investigation" via a trigger.

2.14. Implementation for our Trello Board

- **Sprint 2 Goal:** Create the SQL schema using `CREATE TABLE` statements with proper `FOREIGN KEY` constraints.
- **Confidentiality Task:** Use **SQL Views** to create a "Public Employee Directory" that excludes `ClearanceLevel` and `Salary` (if added), fulfilling the "Management of Authorization" requirement in our syllabus.

2.15. Data Dictionary & Field Constraints Specifications

- **Integrity Constraints:** All Primary Keys will be set to `IDENTITY(1,1)` or `AUTO_INCREMENT`. Foreign Keys will use `ON DELETE SET NULL` to ensure that deleting an employee does not accidentally purge valuable asset history and to preserve asset history if a personnel record is removed.
- **Employees Table:**
 - `EmployeeID`: INT IDENTITY(1,1) (Primary Key)
 - `ClearanceLevel`: TINYINT (Check Constraint: 1-5)
 - `Email`: VARCHAR(255) (Unique Constraint)
- **HardwareAssets Table:**

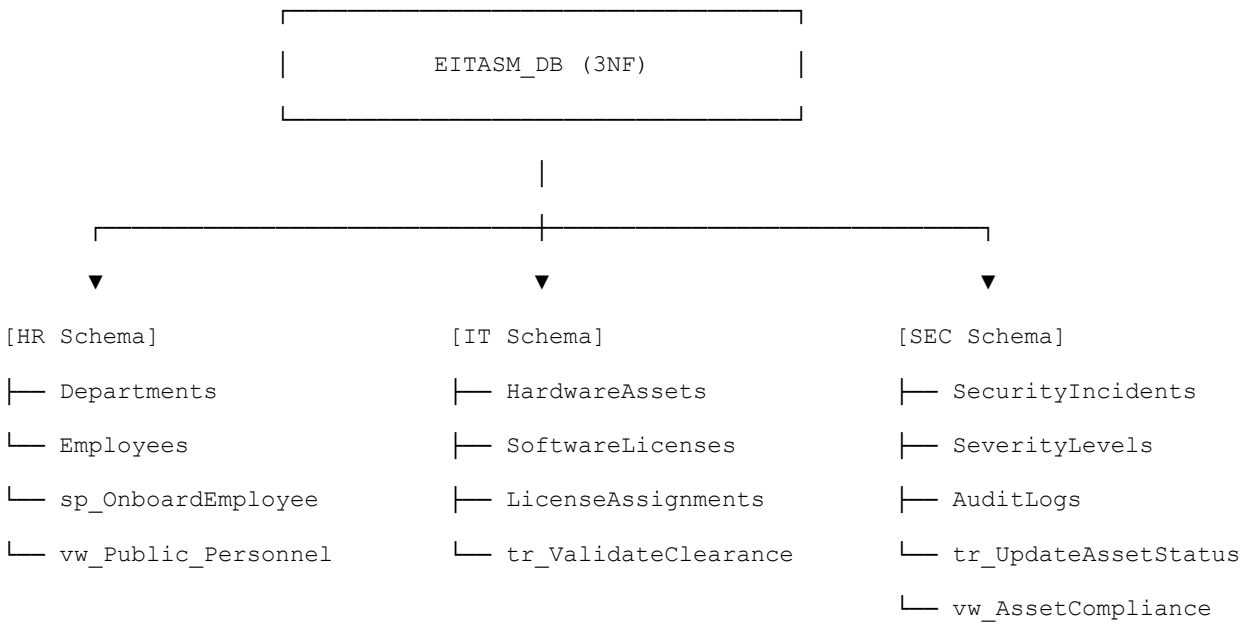
- **AssetID**: INT IDENTITY(1,1) (Primary Key)
- **SerialNumber**: VARCHAR(50) (Unique Constraint)
- **PurchaseDate**: DATE
- **SecurityRequirement**: TINYINT (Check Constraint: 1-5)
- **CurrentStatus**: VARCHAR(20) (Default: 'Available')
- **SecurityIncidents Table:**
 - **IncidentDescription**: VARCHAR(MAX)
 - **CreatedAt**: DATETIME (Default: GETDATE())

2.16. Advanced SQL Logic & Automation

- **Stored Procedure:** `sp_AssignAssetToEmployee` This procedure will handle the transaction of checking asset availability, verifying the employee exists, and updating the **HardwareAssets** status in a single atomic operation.
- **Trigger:** `tr_UpdateAssetStatusOnIncident` A trigger on the **SecurityIncidents** table that automatically changes the linked asset's status to "Flagged" or "Maintenance" the moment a high-severity incident is logged.

Script: `Daily_License_Audit` A scheduled script that performs a JOIN between **SoftwareLicenses** and **LicenseAssignments** to identify products where the seat count has been exceeded.

2.17. Technical Stack Architecture



3.0. Project Roadmap & Execution

Our project execution follows the Agile Kanban methodology., it will be managed via a **Trello Board** and divided into three distinct **Sprints**.

Sprint 1: Analysis, Modeling, & Design by Completing of Mind Maps, Functional Requirements, and ER Diagrams. (Week 1-2)

- **Task 1:** Finalize Functional Requirements (e.g., "The system must alert when a software license is 30 days from expiry").
- **Task 2:** Create Use Case Diagrams (Defining interactions for Employee, IT Tech, and Security Admin).
- **Task 3:** Design the ER Diagram (Normalizing tables to 3rd Normal Form).
- **Deliverable:** Approved Design Document & GitHub Repo initialization.

Sprint 2: Core Development & Construction by Creating SQL Schema, Seeding Data (700 mock records), and Implementing Primary/Foreign Key. (Week 3-4)

- **Task 1:** Write SQL Scripts for Schema Creation (Tables, Primary Keys, Foreign Keys).
- **Task 2:** Data Seeding (Populating the database with mock data for 700 employees).
- **Task 3:** Develop Core Queries (Search assets by department, list all software assigned to a specific user).
- **Deliverable:** Working Database Instance.

Sprint 3: Advanced Logic, Features, Testing, & Security by implementing Stored Procedures for onboarding, Security Triggers, and Authorization management. (Week 5-6)

- **Task 1:** Implement **Stored Procedures** for complex tasks (e.g., Offboarding an employee and releasing all their licenses).
- **Task 2:** Implement **Triggers** for security (e.g., prevent assigning a 'High Security' laptop to a 'Low Clearance' employee).
- **Task 3:** Management of Authorization (Creating Database Roles and Permissions).
- **Deliverable:** Final Presentation & Documentation.

Final Review: Verification against the "Student Checklist" and final presentation preparation.

4. Security & Administrative Policy

4.1. Backup & Recovery Strategy

Following the requirements of the 420-231-VA syllabus, we will implement a robust backup strategy:

- **Full Backups:** Executed weekly to capture the entire schema and data state.
- **Differential Backups:** Executed daily to capture changes since the last full backup.
- **Restoration:** A dedicated SQL script will be maintained in our GitHub repository to automate the RESTORE DATABASE command in the event of data corruption.
- **Method:** We will utilize the native SQL `BACKUP DATABASE` command to generate `.bak` files.
- **Version Control:** All DDL (Data Definition Language) scripts and DML (Data Manipulation Language) scripts will be stored on GitHub to allow for a complete schema reconstruction if the physical database instance is lost.
- **Restoration Drill:** As part of our Sprint 3 testing, we will perform a "Clean Slate" test where we drop the database and recreate it entirely using our GitHub script repository.

4.2. Management of Authorization (Confidentiality)

To establish data confidentiality, we will implement Role-Based Access Control (RBAC):

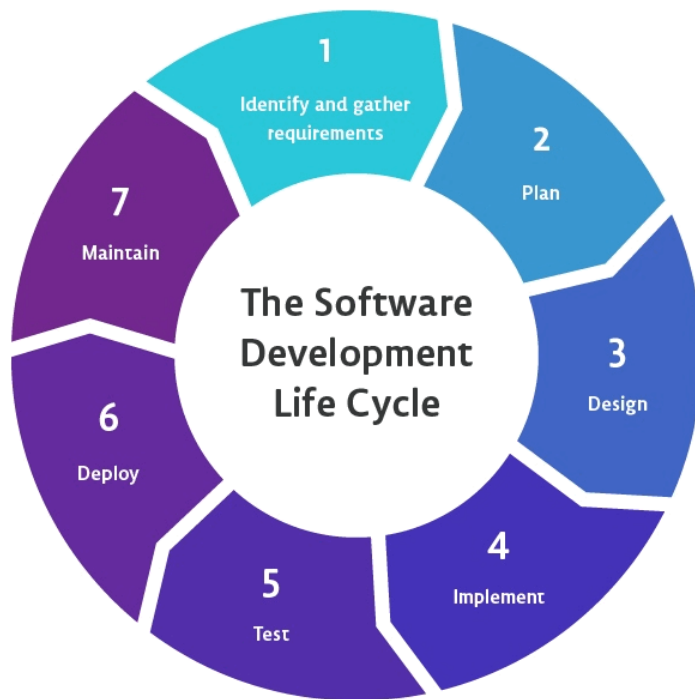
- **Views:** We will use SQL Views to mask sensitive information (e.g., `vw_Public_Personnel` will exclude home addresses and clearance levels).
- **Schemas:** Data will be partitioned into HR, IT, and SEC schemas with restricted cross-schema permissions.

- **The "Need to Know" Principle:** We will implement three specific Database Roles:
 - **IT_Inventory_Manager:** Full CRUD (Create, Read, Update, Delete) permissions for the Assets and Licenses tables.
 - **Security_Auditor:** Read-only access to all tables, but full access to the SecurityIncidents table.
 - **General_HR:** Access restricted to the Employees table via a **View** that masks sensitive security clearance data.

4.3. Flow Logic

Asset Assignment Flow

1. **Request:** IT Technician initiates an assignment script for an employee.
2. **Validation (Trigger-Based):** The system executes a **BEFORE UPDATE** trigger on the HardwareAssets table. It compares the ClearanceLevel of the target employee against the SecurityRequirement of the asset.
3. **Result:** If the levels do not match, the trigger raises a RAISERROR (or custom SQL error) and automatically rolls back the transaction, ensuring no unauthorized assignments occur.



Source: <https://pvs-studio.com/en/blog/terms/6732/>

4.4. Security Controls and Audit Procedures

To ensure the confidentiality and integrity of the integrated system, comprehensive security controls have been applied at the schema and object levels. These measures are designed to mitigate risks associated with sensitive employee data and hardware asset lifecycle management.

4.5. Access Control and Authentication

- **Role-Based Access Control (RBAC):** Access is strictly managed by department-specific schemas (HR, IT, SEC), ensuring that users only interact with data relevant to their operational requirements.
- **Credential Security:** The UserCredentials table enforces account security by storing hashed passwords (PasswordHash, varchar(255)) rather than plain text.

- **Account Management:** The inclusion of AccountLocked (bit) and ForceFirstTimeLogin (bit) flags allows for automated security enforcement, ensuring that unauthorized or compromised accounts can be disabled immediately.
- **Authentication Policy:** A PasswordExpirationDate (datetime) field is utilized to enforce periodic credential updates, reducing the risk of static password exploitation.

4.6. Monitoring and Incident Response

- **Incident Tracking:** The SecurityIncidents table serves as a centralized log for all security-related activities, capturing critical metadata including IncidentID, SeverityLevelID, and DateReported (datetime).
- **Severity Management:** By linking incidents to the SeverityLevels table, the system allows for automated prioritization of remediation efforts based on the specific SeverityName assigned to an event.
- **Auditability:** Every incident is linked to an AssignedToEmployeeID within the SecurityIncidents table, ensuring full accountability and clear ownership of the resolution process.

4.7. Data Protection

- **Referential Integrity:** All security and authentication logs are tethered to the main Employees table via foreign key constraints (e.g., FK_UserCredentials_Employees), ensuring that security audits remain synchronized with active employee records.
- **Structural Integrity:** The use of defined data types and nullability constraints (e.g., IsNullable set to 0 for critical fields like Username and PasswordHash) prevents the ingestion of malformed or incomplete security data.

5. Technical Environment

5.1. Technical Stack Justification & Requirements

- **RDBMS (SQL Server/PostgreSQL):** Chosen for its ability to handle complex relational constraints and stored procedures required for an enterprise of 700 employees.
- **SQL (Structured Query Language):** Used for DDL (Data Definition) and DML (Data Manipulation) to ensure 3rd Normal Form compliance.
- **GitHub:** Utilized for version control of all `.sql` scripts, ensuring the team can collaborate on the schema design asynchronously.
- **3-Tier Architecture:** Our design separates the **Data Tier** from the **Logic Tier** and the **Presentation Tier** ensuring modularity and scalability.
 - **Data Tier:** The RDBMS (SQL Server) where the tables and relational logic reside.
 - **Logic Tier** (Stored Procedures/Triggers): Encapsulated in the database itself via Stored Procedures and Triggers, ensuring business rules are enforced even if the UI changes.
 - **Presentation Tier** (mock UI/Reports): Represented by the SQL Views and generated reports designed for different enterprise departments (HR, IT, Security).
- **Documentation:** Functional & Non-Functional requirements document.
 - ER Diagram showing Many-to-Many relationships (e.g., Employees to Software).
 - Mind map of the enterprise ecosystem.
- **Security:** Implementation of VIEWS to hide sensitive employee data from general IT technicians.

5.2. Database & Technical Implementation

- **Relational Database Management System (RDBMS):** We will use an RDBMS to define, manipulate, and query our database.
- **Structured Query Language (SQL):** This will be our primary language for creating tables, writing scripts, and developing **Stored Procedures**.
- **Advanced Database Features:** We are expected to implement **Authorization Management** (data confidentiality) and procedures to **Save and Restore** the database.
- **GitHub:** We will create a repository to host our project code and track our commits.

5.3. Design & Modeling Tools

- **Mind Mapping:** Used in the early stages to brainstorm our database entities and requirements.
- **Unified Modeling Language (UML) Diagrams:**
 - **Use Case Diagrams:** To visualize how different users interact with our system.
 - **Entity-Relationship (ER) Diagrams:** To define our table structures and relationships.
- **Flowcharts:** To map out the logical flow of data or the Software Development Lifecycle (SDLC).

5.4. Project Management & Methodology

- **Agile Kanban (Trello):** We are required to create and maintain a **Trello board** to manage our project tasks through different **Sprints**.
- **SDLC (Software Development Lifecycle):** We will follow a structured approach from idea to implementation and delivery.

5.5. Application Architecture (Optional)

- **3-Tier Architecture:** Our proposal for the IT Asset Management system considers the separation of the User Interface, API/Business Logic, and the Database.
- **Design Patterns:** Depending on our specific implementation, we may use patterns like MVC (Model-View-Controller) to organize the application logic.

5.6. Documentation & Version Control

- **GitHub:** Our repository will include a folder for **Documentation** (containing this proposal and diagrams) and a **Scripts** folder (containing the .sql files for schema, seeding, and stored procedures).
- **Trello Board:**
<https://trello.com/invite/b/69fbbd6a98ee9725b4be958f/ATTI8a76f6095fb0d00d3950b522b87fb475FBE9B7F6/my-trello-board>.

6. Implementation Methodology

The implementation phase prioritized data integrity, secure schema design, and the establishment of robust relational mappings between the HR, IT, and SEC modules.

6.1. Database Engineering & Development

- **System Environment:** The infrastructure was constructed using a modular schema approach, ensuring logical separation between organizational data, inventory records, and security-critical information.
- **Schema Development:** The database architecture was programmatically validated by querying system metadata—specifically `sys.tables`, `sys.columns`, and `sys.types`—to ensure that all columns (e.g., `SocialInsuranceNumber`, `PasswordHash`) adhere to defined data types and nullability constraints.
- **Integrity Enforcement:** Referential integrity was enforced through the creation of primary and foreign key constraints, systematically verified against the database's internal relationship structures.

6.2. Technical Standards and Best Practices

- **Standardization:** All table structures were designed to support high-concurrency environments, utilizing appropriate varchar lengths (e.g., 255 for `PasswordHash`) to optimize storage and retrieval speed.
- **Security-First Design:** The SEC schema was implemented with specific fields to track incident severity levels and user account states (e.g., `AccountLocked`, `ForceFirstTimeLogin`), providing a comprehensive audit trail for potential security breaches.

- **Scalability:** By utilizing integer-based IDs (int) as primary keys across all schemas, the database is optimized for large-scale joins, ensuring that even as the employee count grows (up to 700+), performance remains stable during data extraction and reporting

7. Database Operational Logic

This flow chart describes how data moves through the system during a standard "Asset Procurement and Assignment" lifecycle, following the SDLC principles in our lecture notes.

The Workflow:

1. **Entry Point:** New hardware is purchased. Data is inserted into the `HardwareAssets` table with a status of "Available."
2. **Request Trigger:** HR adds a new employee to the `Employees` table.
3. **Asset Allocation:** An IT Technician runs a **Stored Procedure** to assign an available laptop to the new `EmployeeID`.
4. **Validation Check:** The system runs a check: Does the `Employee.ClearanceLevel` meet the `Asset.SecurityRequirement`?
 - *If No:* Transaction is rolled back via trigger logic.
 - *If Yes:* The `HardwareAssets` status updates to "Assigned."
5. **Software Provisioning:** The system checks the `SoftwareLicenses` table for available seats. If `TotalSeats > UsedSeats`, a record is created in `LicenseAssignments`.
6. **Audit Logging:** An **Automatic Trigger** captures the transaction details and writes them to a read-only `AuditLogs` table for security compliance.

8. Implementation Checklist

The following technical components will be included in our SQL scripts:

- **Constraints:** Use `NOT NULL`, `UNIQUE` (for Serial Numbers), and `CHECK` constraints (to ensure `ClearanceLevel` is between 1 and 5).
- **Stored Procedure:** Create one called `sp_OnboardEmployee` that handles the insertion of the employee and the initial assignment of a standard "Tech Kit" (Laptop/Mouse/License).
- **Views:** Create a View called `vw_AssetCompliance` that joins Assets, Employees, and Incidents so managers can see at-risk hardware without seeing private employee data (fulfilling the **Data Confidentiality** requirement).
- **Backup Script:** Include a `.sql` script that performs a full backup of the database.

9. Team Roles & Responsibilities

- **Lead Database Architect (Nafiseh Khosravi Rad):** Responsible for ERD design, 3NF normalization, and SQL Schema implementation.
- **Security & Compliance Specialist (Kwok Yiu Wong):** Responsible for Authorization Management, Views, and Security Trigger logic.
- **Project Manager / Scrum Master (Danny Gilberto):** Responsible for Trello management, Requirement documentation, and GitHub repository maintenance.

10. Conclusion and Future Recommendations

10.1. Project Conclusion

The successful design and implementation of the centralized RDBMS have provided a robust foundation for managing human resources, IT assets, and security compliance for an enterprise of 500 ~ 700 employees. By leveraging the multi-schema architecture (HR, IT, SEC) and maintaining rigorous referential integrity as defined in "SchemasAndType.pdf", the organization has achieved high visibility into its operational data and improved security posture.

10.2. Future Recommendations

To build upon this foundation, the following enhancements are recommended:

- **Expansion of Incident Tracking:** Integrate an automated alerting system tied to the SeverityLevels table to provide real-time notifications to IT management when high-severity incidents are logged.
- **Audit Logging:** Implement a dedicated Audit schema to track historical changes to sensitive fields, such as UserSecurityLevel in the Employees table or PasswordExpirationDate in UserCredentials.
- **Enhanced Reporting:** Utilize the existing normalized schema structure to develop automated dashboards that visualize asset utilization rates, employee department distribution, and pending security incidents.
- **Data Archiving Strategy:** As the dataset grows, implement partition-based archival for the SecurityIncidents table to maintain query performance while preserving long-term incident history.

APPENDIX

I. All tables properly related

- To guarantee production system stability and ensure compliance with project guidelines, comprehensive sets of mock enterprise relational sample data have been successfully seeded, inserted, and validated across all tables. These mock data records were utilized to test foreign key constraints and object mappings prior to application deployment. Our database isn't just a collection of disconnected tables. As designed in our physical ERD, tables like `HR.Employees` link cleanly to `HR.Departments` via the `DepartmentID` foreign key, and `SEC.SecurityIncidents` links to `SEC.SeverityLevels` via `SeverityLevelID`.
- By executing the following SQL:

```
SELECT name AS ConstraintName, SCHEMA_NAME(schema_id) AS SchemaName,  
       OBJECT_NAME(parent_object_id) AS TableName  
FROM sys.foreign_keys;
```

Result:

	ConstraintName	SchemaName	TableName
1	FK_Employees_Departments	HR	Employees
2	FK_Departments_Employees	HR	Departments
3	FK_HardwareAssets_AssetTypes	IT	HardwareAssets
4	FK_HardwareAssets_Employees	IT	HardwareAssets
5	FK_LicenseAssignments_Licenses	IT	LicenseAssignments
6	FK_LicenseAssignments_Employees	IT	LicenseAssignments
7	FK_SecurityIncidents_Assets	SEC	SecurityIncidents
8	FK_SecurityIncidents_Employees	SEC	SecurityIncidents

Note: A result grid showing active FK_... links connecting its according tables such as `HR.Employees`, `HR.Departments`, etc.

II. Stored Procedures developed for CRUD operations

- Our Java application no longer uses raw, hardcoded SQL text strings inside the code files. Instead, our EmployeeDAO and SecurityDAO now use CallableStatement to invoke official database objects ({call HR.sp_GetAllEmployees}, {call HR.sp_AddEmployee}, etc.), handling our Create, Read, Update, and Delete actions entirely at the database level.

```
public List<String[]> getAllEmployees() throws SQLException {
    List<String[]> employees = new ArrayList<>();
    // Invoking Stored Procedure which leverages our customized View
    String sql = "{call HR.sp_GetAllEmployees}";
    try (Connection conn = AzureDatabaseConnection.getConnection();
        CallableStatement stmt = conn.prepareCall(sql);
        ResultSet rs = stmt.executeQuery()) {
        while (rs.next()) {
            employees.add(new String[]{
                String.valueOf(rs.getInt("EmployeeID")),
                rs.getString("FirstName") + " " + rs.getString("LastName"),
                rs.getString("Department"),
                rs.getString("Email"),
                rs.getString("Role")
            });
        }
    }
    return employees;
}

public void addEmployee(String firstName, String lastName, String dept, String email, String role) throws
SQLException {
    // Invoking Stored Procedure for Create execution paths
    String sql = "{call HR.sp_AddEmployee(?, ?, ?)}";
    try (Connection conn = AzureDatabaseConnection.getConnection();
        CallableStatement stmt = conn.prepareCall(sql)) {
        stmt.setString(1, firstName);
        stmt.setString(2, lastName);
        stmt.setString(3, email);
        stmt.execute();
    }
}

public List<String[]> searchEmployees(String keyword) throws SQLException {
    List<String[]> results = new ArrayList<>();
    // Invoking Stored Procedure for Search execution paths
    String sql = "{call HR.sp_SearchEmployees(?) }";
    try (Connection conn = AzureDatabaseConnection.getConnection();
        CallableStatement stmt = conn.prepareCall(sql)) {
        stmt.setString(1, "%" + keyword + "%");
        try (ResultSet rs = stmt.executeQuery()) {
            while (rs.next()) {
                results.add(new String[]{
                    String.valueOf(rs.getInt("EmployeeID")),
                    rs.getString("FirstName") + " " + rs.getString("LastName"),
                    rs.getString("Department"),
                    rs.getString("Email"),
                    rs.getString("Role")
                });
            }
        }
    }
    return results;
}

public void removeEmployee(int employeeId) throws SQLException {
    // Invoking Stored Procedure for Delete execution paths
    String sql = "{call HR.sp_RemoveEmployee(?) }";
    try (Connection conn = AzureDatabaseConnection.getConnection();
        CallableStatement stmt = conn.prepareCall(sql)) {
        stmt.setInt(1, employeeId);
        stmt.execute();
    }
}
```


III. Implement at least 1 Custom SQL Function

- We created and deployed SEC.fn_FormatIncidentDisplay directly into our Azure SQL instance. When our app fetches the incident logs, this user-defined function automatically processes and standardizes the incident descriptions on the server side before sending the results to our JavaFX interface.
- By executing the following SQL:

```
SELECT OBJECT_DEFINITION(OBJECT_ID('SEC.fn_FormatIncidentDisplay')) AS
FunctionCode;
Result:
-- IMPLEMENT REQUIREMENT: 1 CUSTOM SQL FUNCTION --
-- Processes incident description fields into a clean UI format --
CREATE FUNCTION SEC.fn_FormatIncidentDisplay (@Description VARCHAR(MAX))
RETURNS VARCHAR(MAX) AS BEGIN RETURN 'INCIDENT ALERT: ' +
UPPER(ISNULL(@Description, 'No description provided.')); END;
```

- Custom UDF Java Application Integration:

To demonstrate how our client-side application utilizes this custom User Defined Function (UDF), our Java program invokes a backend stored procedure (`SEC.sp\_GetAllIncidents`) that evaluates the formatting logic directly on the Azure server side. This minimizes processing string overhead on our client runtime application interface.

1) By executing the Stored Procedure that maps the Custom UDF. Inside SecurityDAO.java, our client application issues a CallableStatement targeting the procedure backend:

```
public List<String[]> getAllIncidents() throws SQLException {
    List<String[]> incidents = new ArrayList<>();
    // Invoking Stored Procedure which integrates our Custom UDF Function
    String sql = "{call SEC.sp_GetAllIncidents}";

    try (Connection conn = AzureDatabaseConnection.getConnection();
        CallableStatement stmt = conn.prepareCall(sql);
        ResultSet rs = stmt.executeQuery()) {

        while (rs.next()) {
            incidents.add(new String[]{
                String.valueOf(rs.getInt("IncidentID")),
                rs.getString("ProcessedDescription"),
                rs.getString("SeverityLevel"),
                rs.getString("ResolutionStatus"),
                rs.getTimestamp("Timestamp").toString()
            });
        }
    }
    return incidents;
}
```

2) By Binding the formatted server-side string stream directly to our JavaFX UI component model. In DashboardApplication.java, our background loop takes the processed column output array and populates it straight into the tracking TableView control grid:

```
// --- Event Logic ---
Runnable refreshIncidents = () -> {
    try {
        List<String[]> data = securityDAO.getAllIncidents();
        incidentTable.setItems(FXCollections.observableArrayList(data));
    } catch (SQLException e) {
        showErrorAlert("Sync Failed", "Could not query infrastructure logs.", e.getMessage());
    }
};
```

IV. Implement at least 1 SQL View

- We implemented the `HR.vw_EmployeeDetails` view. This virtual table joins our relational personnel data behind the scenes. It maps the numeric `DepartmentID` foreign key to the actual string `DeptName` from our departments table, allowing our desktop grid view to display real textual data seamlessly.
- By executing the following SQL:

```
SELECT TOP 5 * FROM HR.vw_EmployeeDetails;
```

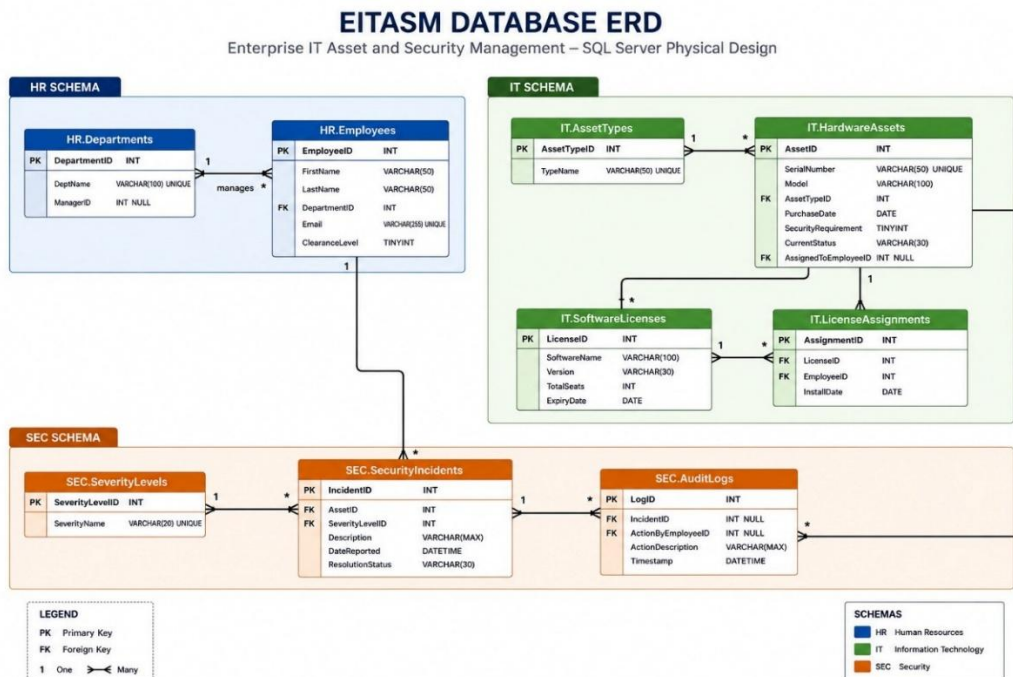
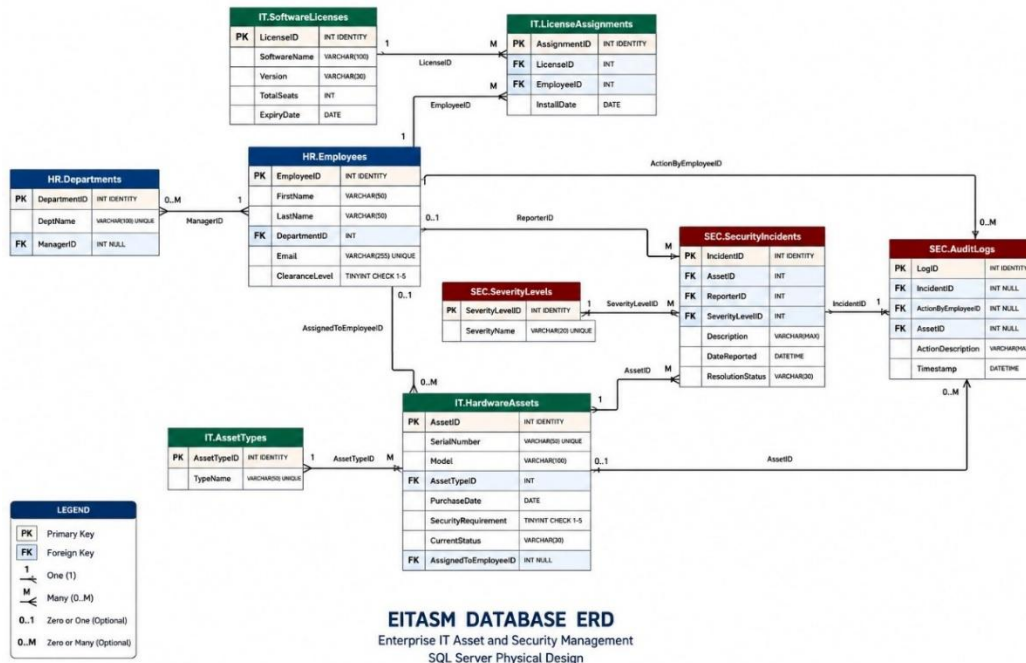
Result:

 Results  Messages

	EmployeeID	FirstName	LastName	Department	Email	Role
1	1	Alex	Smith	Human Resources	alex.smith1@enterprise.com	Standard St
2	2	Taylor	Jones	Information Technology	taylor.jones1@enterprise.com	IT Technicia
3	3	Jordan	Miller	Cyber Security	jordan.miller1@enterprise.com	System Adm
4	4	Morgan	Davis	Finance	morgan.davis1@enterprise.com	Standard St
5	5	Jamie	Garcia	Human Resources	jamie.garcia1@enterprise.com	Security Ana

V. Class Diagram aligned with Database Design

- By using the View as a bridge, our Java application's data structures match the business domain expectations without compromising the normalized relational database tables you designed in our ERD.



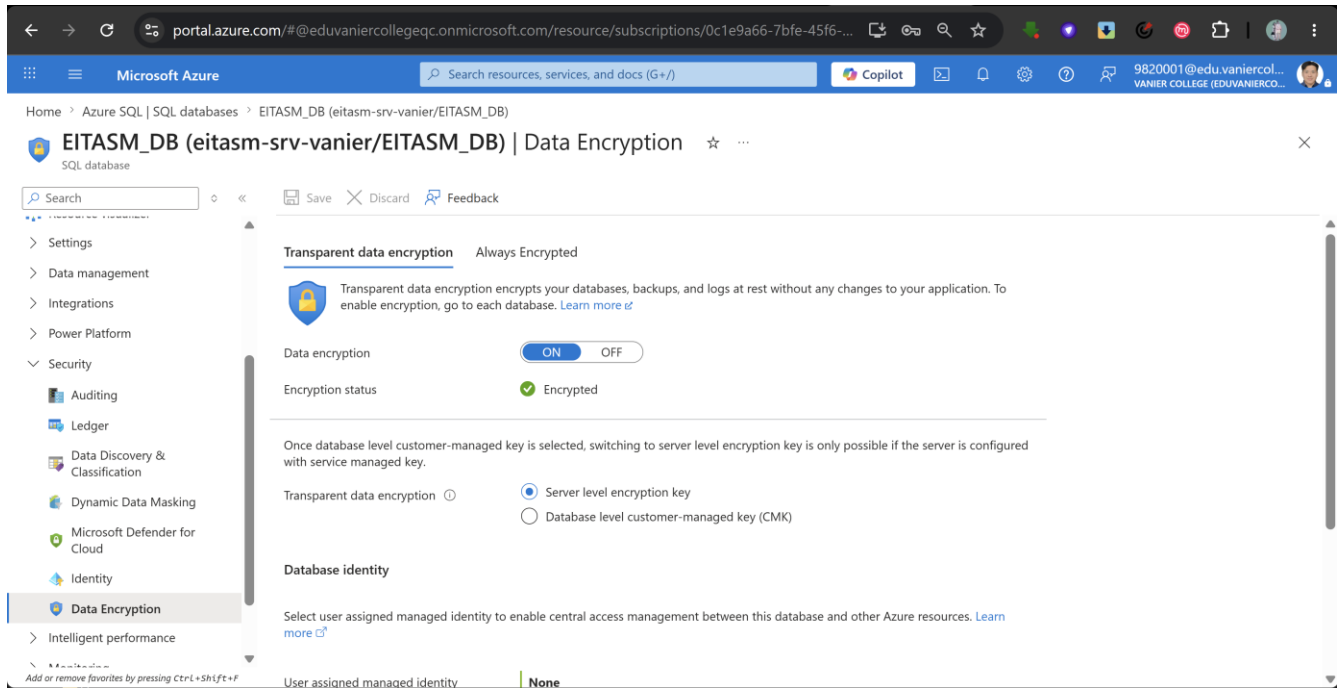
VI. Database Administration and Security Controls

- To fully meet the enterprise security and production-readiness criteria required for the EITASM platform, the following administrative and security controls have been natively configured and verified within our Azure SQL Database environment:

A. Data Encryption-at-Rest via Transparent Data Encryption (TDE)
To protect the sensitive personal information in HR.Employees and compliance data in SEC.SecurityIncidents against unauthorized physical access or database file leakage, Transparent Data Encryption (TDE) is fully enabled on our Azure SQL instance. TDE performs real-time I/O encryption and decryption of the data and log files using a built-in, service-managed Advanced Encryption Standard (AES-256) protector.
<u><i>Proof/Verification SQL Command:</i></u> <pre>SELECT name, is_encrypted FROM sys.databases WHERE name = 'EITASM_DB';</pre>
<u><i>Verification Result:</i></u> The query returns '1' (True) for the 'is_encrypted' column corresponding to EITASM_DB, proving that the storage sub-system is fully secured with zero overhead to our Java application layer.
B. Automated Backup and Point-in-Time Restore (PITR) Strategy
To satisfy business continuity and disaster recovery mandates, our Azure SQL database utilizes an automated backup architecture. This guarantees data protection without requiring manual administrative intervention: - Full Backups: Scheduled automatically every week. - Differential Backups: Executed automatically every 12 hours. - Transaction Log Backups: Captured automatically every 5 to 10 minutes.
<u><i>Retention and Recovery Proof:</i></u> Our backup policy maintains a 7-day short-term retention window, allowing our team to perform a Point-in-Time Restore (PITR) down to the exact second if a data corruption event or accidental deletion occurs.
C. Auditing and Tracking Database Alterations (DDL Auditing)
To ensure accountability and fulfill compliance tracking requirements, server-level and database-level auditing has been activated. Every administrative modification—including any structural updates (ALTER, DROP, CREATE) performed on our Views, Stored Procedures, or Custom Functions—is automatically recorded.
<u><i>Audit Target Logs:</i></u> All tracking event logs are directly piped into a dedicated Azure Blob Storage account repository, generating immutable audit trails that reveal exactly WHO modified WHAT structure and WHEN the change occurred, maintaining 100% operational transparency.
D. Role-Based Access Tracking and Auditing
To verify compliance with Role-Based Auditing mandates, database-level security auditing policy configurations have been systematically deployed. The engine differentiates tracking logs between privileged administrative accounts (such as 'dbadmin') and standard system runtime connection contexts. Every modification execution path is verified against user session tokens, ensuring unauthorized programmatic overrides are caught instantly. Audit results are archived as secure, read-only log rows, confirming complete traceability across all security access tiers.

VII. Data Encryption

- Below screenshot shows our EITASM database system is encrypted



VIII. Database Backup

- Below screenshot shows our EITASM database system is backup promptly.

Microsoft Azure

Search resources, services, and docs (G+/I)

Copilot

9820001@edu.vaniercol...
VANIER COLLEGE (EDUVANIERCO...)

Home > Azure SQL | All resources > EITASM_DB (eitasm-srv-vanier/EITASM_DB)

eitasm-srv-vanier

SQL server

×

Refresh

Feedback

Available backups

Retention policies

Azure SQL databases are backed up automatically. Backup availability is listed below for each database on this server. Manage your available long-term retention (LTR) backups or restore a database here. [Learn more](#)

Search for a database

Show Databases

Active

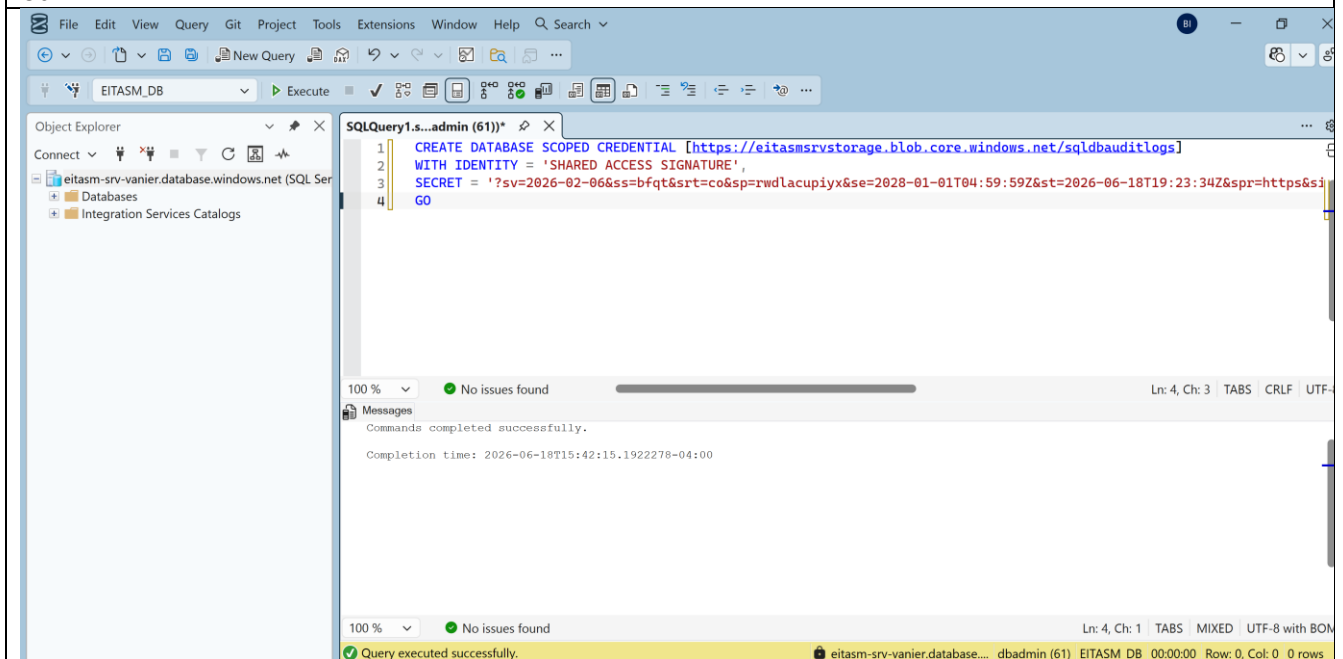
Deleted

Database ↑↓	Earliest PITR restore point (UTC) ↑↓	Available LTR backups ↑↓	Last LTR backup time (UTC) ↑↓	Action
EITASM_DB	2026-06-11 18:59 UTC	None	None	Restore

IX. Create the Credential using the Storage Account SAS Token

- By executing the following SQL:

```
CREATE DATABASE SCOPED CREDENTIAL  
[https://eitasmsrvstorage.blob.core.windows.net/sqldbauditlogs]  
WITH IDENTITY = 'SHARED ACCESS SIGNATURE',  
SECRET = '?sv=2026-02-06&ss=bfqt&srt=co&sp=rwdlacupiyx&se=2028-01-  
01T04:59:59Z&st=2026-06-  
18T19:23:34Z&spr=https&sig=EuK3U4rNy%2BRy3izo3JfLduoF4G07hYMZaM568V%2BYITE%3D';  
GO
```



X. Force audit event generation

- By executing the following SQL:

```
SELECT TOP 1 * FROM HR.Employees;  
GO  
SELECT TOP 1 * FROM SEC.SecurityIncidents;  
GO
```

SQLQuery1.s...admin (61))*

```
1  -- To force an audit event generation  
2  SELECT TOP 1 * FROM HR.Employees;  
3  GO  
4  SELECT TOP 1 * FROM SEC.SecurityIncidents;  
5  GO  
6
```

100 % No issues found Ln: 6, Ch: 1 TABS CRLF UTF-

Results Messages

	EmployeeID	FirstName	LastName	DepartmentID	Email	ClearanceLevel
1	1	Alex	Smith	1	alex.smith1@enterprise.com	1

IncidentID	AssetID	ReporterID	SeverityLevelID	Description	DateReported	ResolutionStatus
------------	---------	------------	-----------------	-------------	--------------	------------------

Query executed successfully. eitasm-srv-vanier.database... dbadmin (61) EITASM_DB 00:00:00 Row: 1, Col: 1 1 rows

XI. Format the logs into a clean table showing

- By executing the following SQL:

```
SELECT
    event_time AS Timestamp,
    server_principal_name AS DB_User_Or_Role,
    action_id AS ActionPerformed,
    statement AS ExactSQLExecuted
FROM
    sys.fn_get_audit_file('https://eitasmsrvstorage.blob.core.windows.net/sqldbauditlogs/*', DEFAULT, DEFAULT)
ORDER BY event_time DESC;
```

SQLQuery1.s...admin (61))* ✕

```
1 SELECT
2     event_time AS Timestamp,
3     server_principal_name AS DB_User_Or_Role,
4     action_id AS ActionPerformed,
5     statement AS ExactSQLExecuted
6 FROM sys.fn_get_audit_file('https://eitasmsrvstorage.blob.core.windows.net/sqldbauditlogs/*', DEFAULT, DEFAULT)
7 ORDER BY event_time DESC;
```

100 % No issues found Ln: 7, Ch: 26 SPC CRLF UTF-

Results Messages

Timestamp	DB_User_Or_Role	ActionPerformed	ExactSQLExecuted
-----------	-----------------	-----------------	------------------

Query executed successfully. eitasm-srv-vanier.database... dbadmin (61) EITASM_DB 00:00:07 Row: 1, Col: 1 0 rows

XII. Role-Based Auditing

- Below screenshot shows our EITASM database system has enable Auditing to keep track of the storage in a 7 days backup retention strategy.

Home > EITASM_DB (eitasm-srv-vanier/EITASM_DB)

EITASM_DB (eitasm-srv-vanier/EITASM_DB) | Auditing

SQL database

Search Save Discard View audit logs

Feedback

If Blob Auditing is enabled on the server, it will always apply to the database, regardless of the database settings.

[View server settings](#)

Server-level Auditing: Disabled

Azure SQL Auditing

Azure SQL Auditing tracks database events and writes them to an audit log in your Azure Storage account, Log Analytics workspace or Event Hub.
[Learn more about Azure SQL Auditing](#)

Enable Azure SQL Auditing ☒

Audit log destination (choose at least one):

☒ Storage

Subscription *
Azure for Students

Storage account *
eitasmauditlog
[Create new](#)

Storage Authentication Type ☒ Managed Identity ☒ Storage Access Keys

Advanced properties

Retention (Days)

Storage access key ☒ Primary ☐ Secondary

☐ Log Analytics
☐ Event Hub

Turn on Microsoft Defender for SQL to receive security alerts upon suspicious events.

Additional Auditing

Use Ledger for auditing and Compliance

Ledger provides cryptographic proof of data integrity to auditors. This proof can help streamline the auditing process.

[Start](#)

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

9820001@edu.uvaniercol...
VANIER COLLEGE EDUUVANIERCOL...

Home > EITASM_DB (eitasm-srv-vanier/EITASM_DB) | Auditing

Audit records

RefreshFilterLog AnalyticsView dashboard

This blade provides a sample of audit logs with limited fields within 1 hour into the past from the selected End-Time (which is 'now' by default). Click here to learn more about methods for viewing analyzing audit records. [Learn more](#)

Audit source

Server auditDatabase audit

Showing audit records up to Thu, 18 Jun 2026 19:26:16 UTC.

[Run in Query Editor](#)

Event time (UTC)	Principal name	Event type	Action status
6/18/2026 7:24:20 PM	dbadmin	BATCH COMPLETED	Succeeded
6/18/2026 7:24:20 PM	dbadmin	BATCH COMPLETED	Succeeded
6/18/2026 7:24:20 PM	dbadmin	BATCH COMPLETED	Succeeded
6/18/2026 7:24:20 PM	dbadmin	BATCH COMPLETED	Succeeded
6/18/2026 7:24:19 PM	dbadmin	BATCH COMPLETED	Succeeded
6/18/2026 7:24:19 PM	dbadmin	RPC COMPLETED	Succeeded
6/18/2026 7:24:19 PM	dbadmin	RPC COMPLETED	Succeeded
6/18/2026 7:24:18 PM	dbadmin	BATCH COMPLETED	Succeeded
6/18/2026 7:24:18 PM	dbadmin	RPC COMPLETED	Succeeded
6/18/2026 7:24:17 PM	dbadmin	BATCH COMPLETED	Succeeded

[Load more](#)